National Teachers College
629 J Nepomuceno, Quiapo, Manila, 1001 Metro Manila
Bachelor of Science in Information Technology

Final Project/Exam: C# Application Development for Sustainable Development

SOFTWARE DESIGN & ANALYSIS DOCUMENT (SDAD)
ECOINVENT: SMART INVENTORY AND WASTE TRACKER SYSTEM

Aligned with United Nations Sustainable
Development Goal 12: Responsible Consumption and Production

BSIT 3.2

Submitted by:
Arcilla, EJ
Berdejo, Justine Marlowie
Bitara, Peter John
Guico, Karlo Emanuel
King, Allen Marlon

Instructor:
Ms. Justin Louise R. Neypes

Date:
March 20, 2026

I. INTRODUCTION

1.1 Project Overview

The **EcoInvent** system is a desktop-based inventory management application designed to provide an efficient and organized way of managing inventory items, categories, and transactions. The system follows a layered architecture consisting of the User Interface (UI), Business Logic Layer (BLL), and Data Access Layer (DAL), ensuring a structured and maintainable design.

The vision of the system is to help businesses and organizations improve their inventory management processes by reducing manual errors, improving data organization, and increasing operational efficiency.

This project aligns with the **United Nations Sustainable Development Goals (SDGs)**, specifically:

- **SDG 12: Responsible Consumption and Production** – by promoting efficient inventory tracking and reducing waste through proper stock monitoring

1.2 Problem Statement

Many small businesses and organizations still rely on manual or unorganized methods for managing inventory. This leads to several problems such as:

- Overstocking of items
- Expiration of unused or untracked products
- Increased waste generation
- Difficulty identifying low-stock items
- Lack of visibility into inventory trends
- No structured data backup or recovery mechanism

Without an efficient system, inventory management becomes time-consuming, inaccurate, and unreliable.

To address these issues, the EcoInvent system was developed to provide a **centralized, secure, and user-friendly solution** for managing inventory data efficiently.

II. REQUIREMENTS & ANALYSIS

2.1 Functional Requirements

The system shall provide the following functionalities:

- **FR1: User Authentication**
  The system shall allow users to log in using secure credentials verified by the AuthService.

- **FR2: Inventory Item Management**

  The system shall allow users to add, update, delete, and view inventory items.
- **FR3: Category Management**

  The system shall allow users to create and manage item categories.
- **FR4: Inventory Transactions**

  The system shall record stock-in and stock-out transactions.
- **FR5: Data Storage**

  The system shall store and retrieve data using a structured database through AppDbContext.
- **FR6: Input Validation**

  The system shall validate user inputs in the Business Logic Layer before processing.
- **FR7: Logging System**

  The system shall log system activities and errors using a Logger component.

  FR8: User Interface Interaction

  The system shall provide a Windows Forms interface for user interaction.

2.2 Non-Functional Requirements

Performance

- The system should respond to user actions (e.g., add/update item) within **1–2 seconds**
- The system should efficiently handle multiple inventory records without significant delay

Security

- The system must implement **password hashing** for user credentials
- Only authenticated users can access the system
- Sensitive data must be protected from unauthorized access

Robustness

- The system should handle invalid inputs gracefully using validation rules
- The system should not crash when errors occur and must log them using the Logger
- The system should maintain data integrity even during unexpected failures

2.3 Complexity Analysis (Big O Notation)

The core business logic of the EcoInvent system is handled in the InventoryService, particularly in operations such as adding, searching, and updating inventory items.

Example: Searching for an Item

If the system uses a list-based search to find an item:

- The algorithm checks each item one by one
- Time Complexity: O(n)

Meaning:
As the number of items increases, the time it takes to search also increases linearly.

Example: Adding an Item

- Adding an item to a list or database
- Time Complexity: O(1) (constant time, assuming direct insertion)

Example: Updating an Item

- The system first searches for the item → O(n)
- Then updates the item → O(1)

Overall Complexity: O(n)

Summary of Complexity

Operation	Complexity
Add Item	O(1)
Search Item	O(n)
Update Item	O(n)
Delete Item	O(n)

III. DESIGN SPECIFICATION

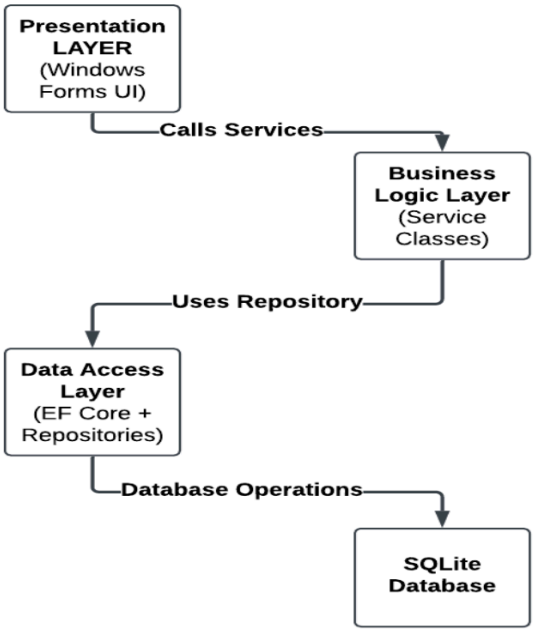
3.1 N-Tier Architecture Diagram

EcoInvent is structured using a Three-Tier (N-Tier) Architecture to organize the system into clear and independent layers. The application is divided into the Presentation Layer, Business Logic Layer, and Data Access Layer, each with its own specific responsibilities. This structure promotes clean code organization, easier maintenance, and controlled communication between components, while supporting the system's inventory and waste-tracking functions.

The Presentation Layer is implemented in C# Windows Forms and is responsible for handling user interactions, capturing inputs, and displaying inventory data and reports. It communicates directly with the Business Logic Layer and does not contain database queries or business computations.

Business Logic Layer processes the core rules of the system, including stock monitoring, expiration checking, and waste calculation. It validates user inputs, performs asynchronous operations when required, and interacts with the Data Access Layer for data persistence.

Data Access Layer manages all database-related operations using Entity Framework Core with SQLite. It performs CRUD operations, handles entity-to-table mapping, and ensures proper data storage and retrieval.

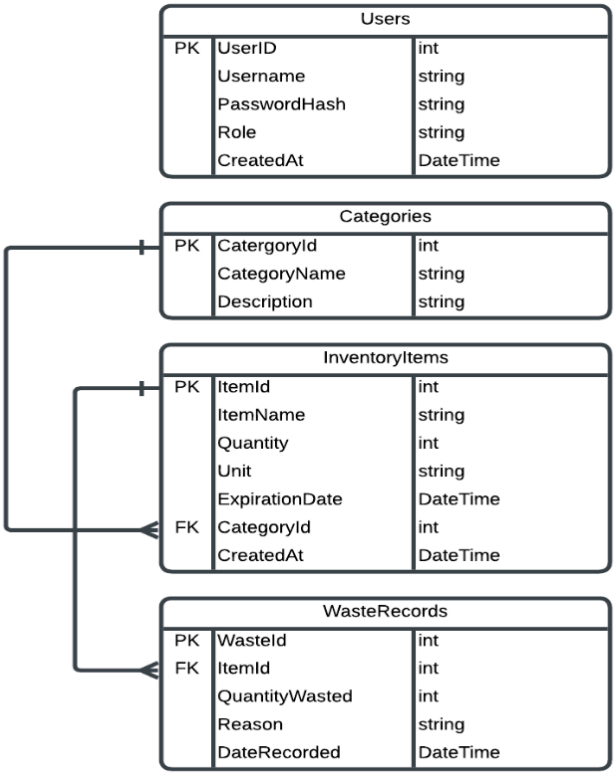


3.1.1 Architectural Flow

The system follows a strict unidirectional data flow:

1. The user interacts with the Presentation Layer (EcoInvent.UI) through Windows Forms.
2. The Presentation Layer calls methods from the Business Logic Layer (EcoInvent.BLL) Service classes.
3. The Business Logic Layer validates inputs, applies business rules, and performs computations.
4. The Business Logic Layer invokes repository methods from the Data Access Layer (EcoInvent.DAL).
5. The Data Access Layer communicates with the SQLite database via Entity Framework Core.
6. Results are returned upward through the layers and displayed in the user interface.

3.2 Entity Relationship Diagram (ERD)



3.3 Use Case & Data Flow Diagrams (DFD)

The Use Case Diagram illustrates the key interactions between the two system actors — Admin and Viewer — and the core system functions.

Actors:

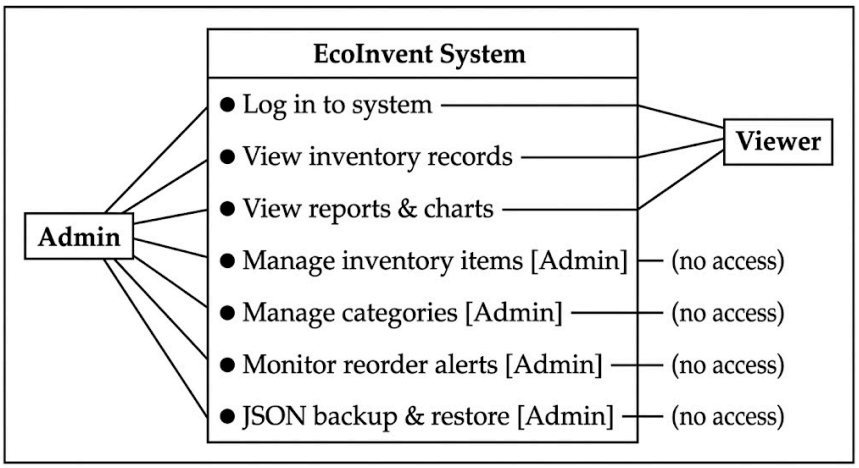
- Admin — has full access to all system features
- Viewer — restricted to read-only access

Use Cases accessible by all users (Admin and Viewer):

- Log in to system
- View inventory records
- View reports and charts

Use Cases accessible by Admin only:

- Manage inventory items (Add, Update, Delete)
- Manage categories
- Monitor reorder alerts
- JSON backup and restore



Data Flow Diagram — Level 0 (Context Diagram)

The Level 0 DFD presents EcoInvent as a single system process that receives inputs from external actors and produces outputs, while interacting with two external data stores.

External Entities: Admin user, Viewer user

Data Flows:

- Admin → System: credentials, item data, category data, backup/restore commands
- System → Admin: session token, reorder alerts, reports, operation results
- Viewer → System: view requests
- System → Viewer: inventory data, charts and reports
- System ↔ SQLite DB: write/read inventory and user data; receive query results
- System ↔ JSON File: export data to file; import data from file

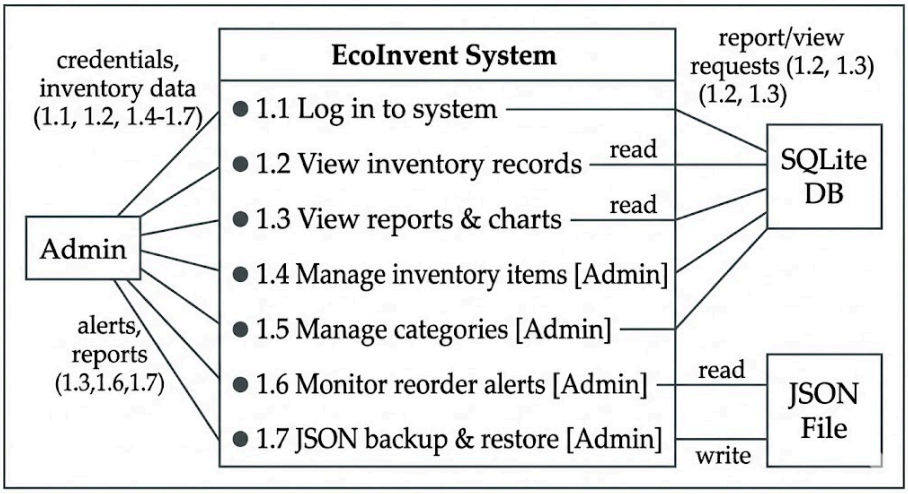


Figure 3.3.2 — DFD Level 0 Context Diagram

Data Flow Diagram — Level 1 (Process Decomposition)

The Level 1 DFD decomposes the EcoInvent system into six sub-processes, five data stores, and all data flows between them.

Data Stores:

- D1 — Users
- D2 — InventoryItems
- D3 — Categories
- D4 — WasteRecords
- D5 — JSON Backup File

Processes:

P1 — Authentication

- Receives credentials from Admin or Viewer
- Reads and writes to D1 (Users) to verify or store user records
- Returns session token and role to the user

P2 — Inventory Management

- Receives item data from Admin
- Performs CRUD operations on D2 (InventoryItems)
- Returns item records to display
- Sends waste triggers to P4 when items expire or are marked wasted

P3 — Category Management

- Receives category data from Admin
- Performs CRUD operations on D3 (Categories)
- Returns category list to the UI

P4 — Waste Tracking

- Receives waste trigger from P2
- Logs waste events to D4 (WasteRecords)
- Returns waste records for reporting

P5 — Report Generation

- Receives view requests from Viewer or Admin
- Reads data from D2 (InventoryItems) and D4 (WasteRecords)
- Returns charts and reports to the user

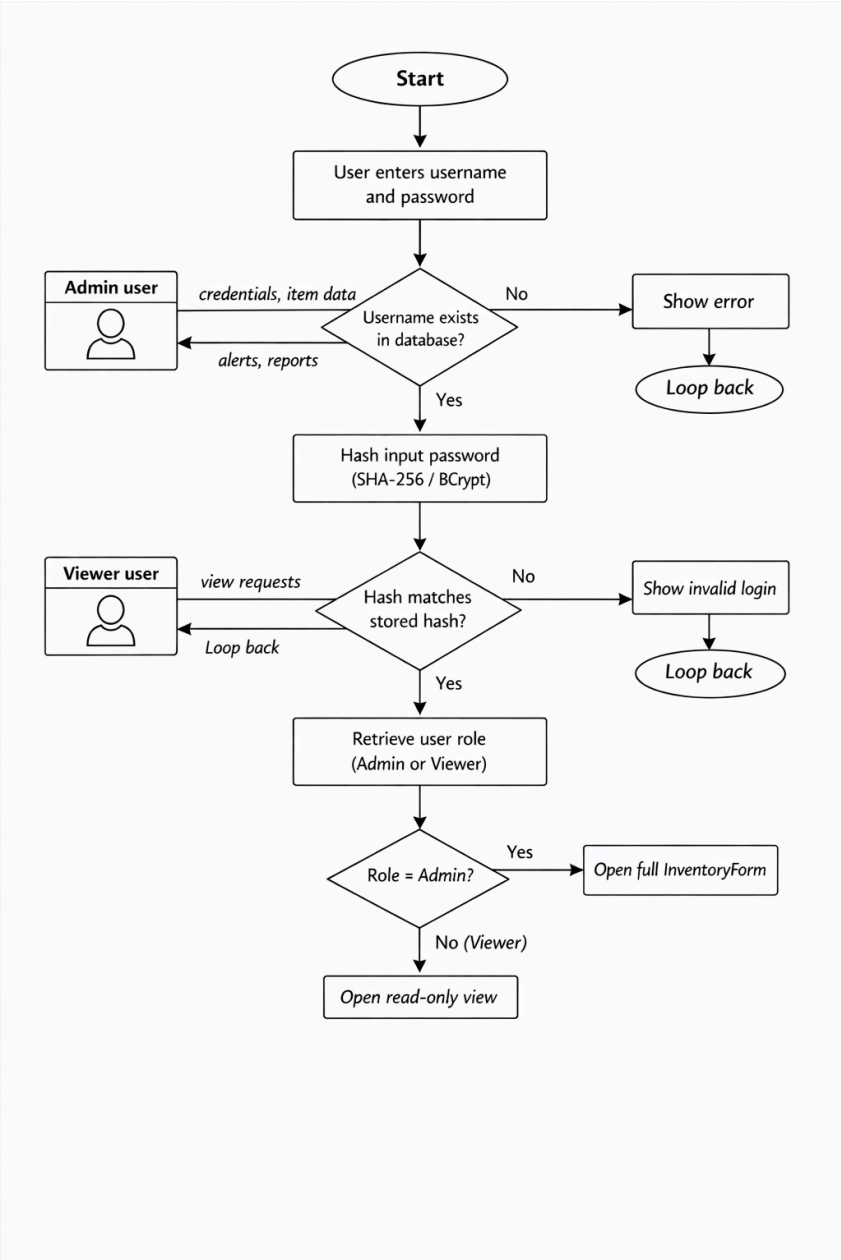
P6 — JSON Backup / Restore

- Receives backup or restore commands from Admin (Admin-only flow)
- Exports serialized data to D5 (JSON Backup File)
- Imports and deserializes data from D5, then writes restored records to D2
- Solid lines indicate standard data flows; dashed lines indicate Admin-only flows

3.4 Algorithm Flowchart

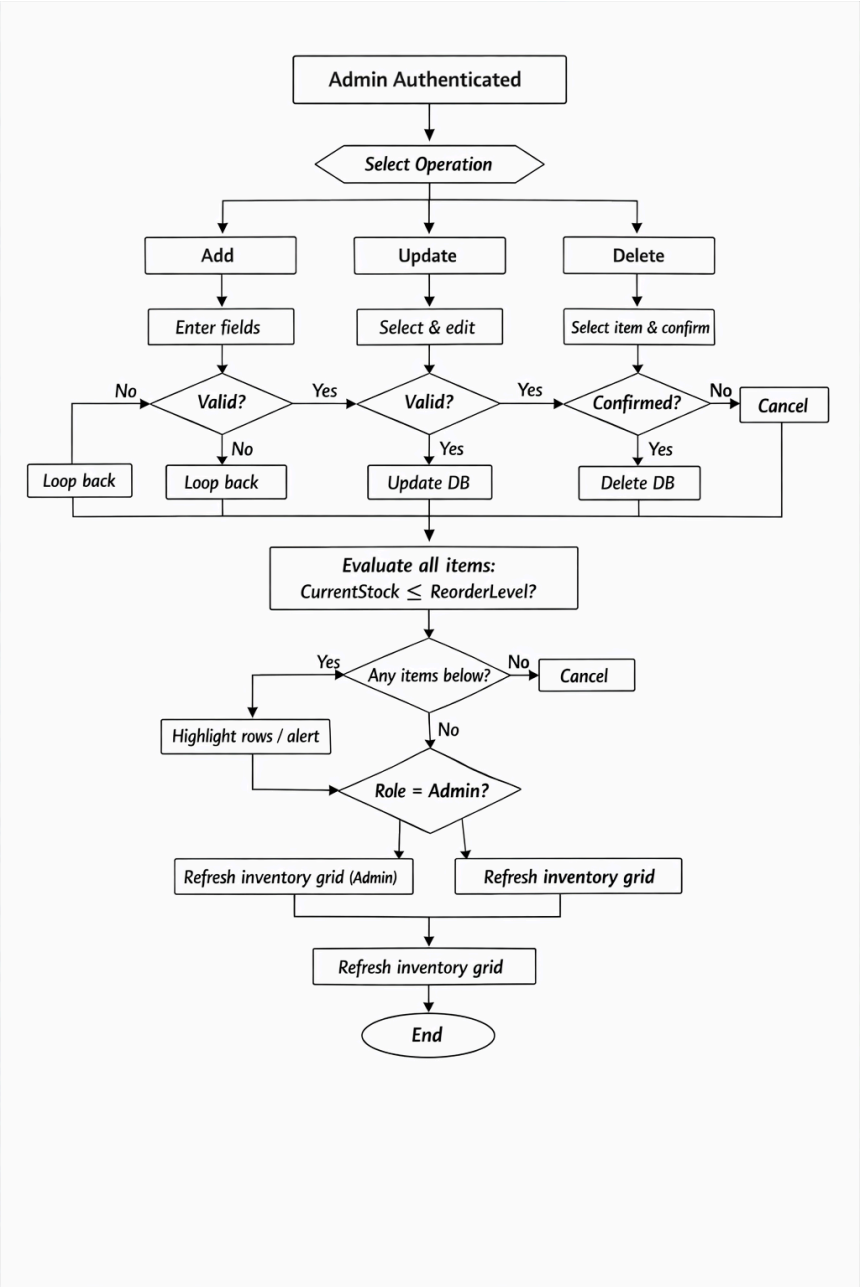
Flowchart 1 — User Authentication

Purpose: Describes the login decision path from credential input to role-based form routing.



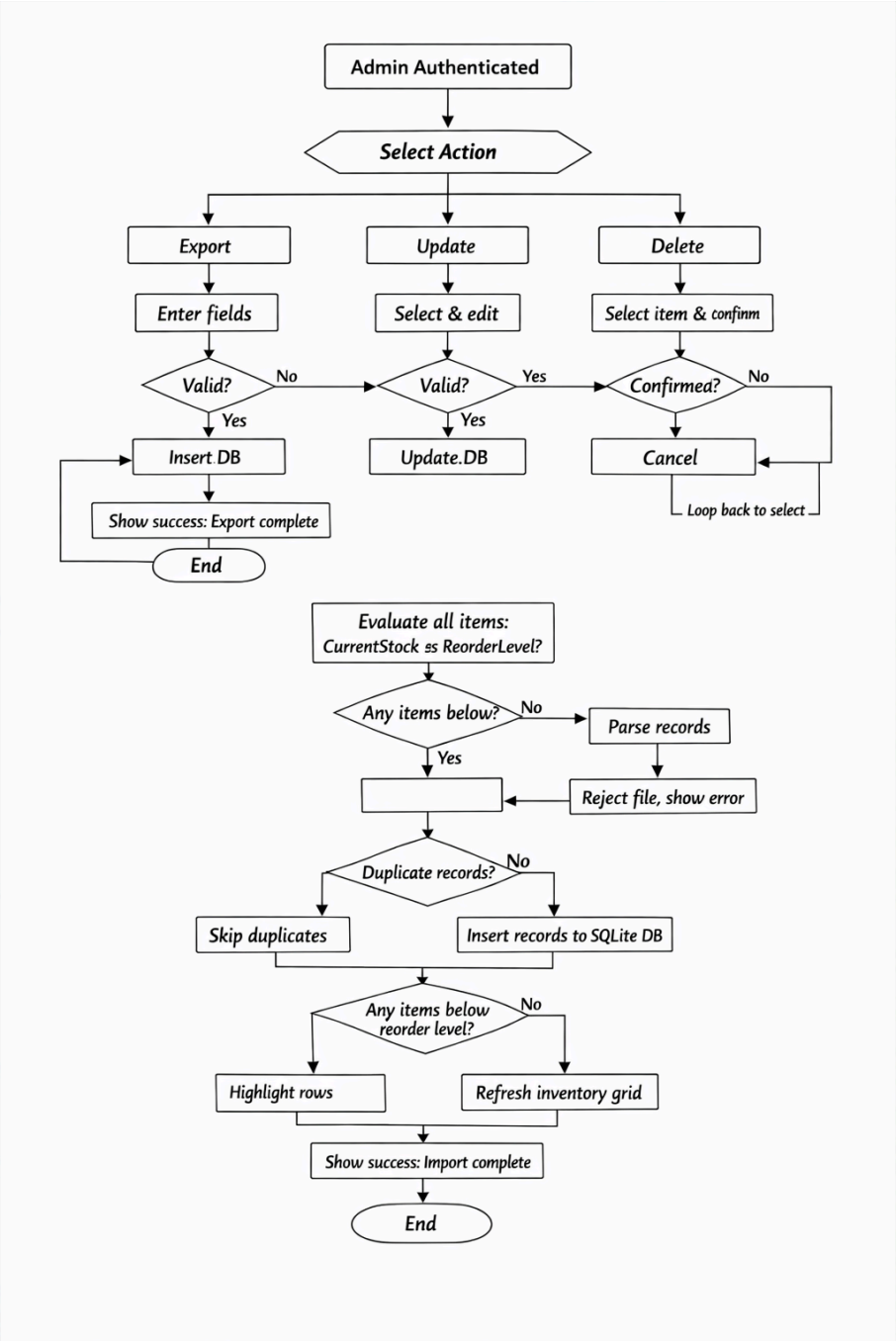
Flowchart 2 — Inventory CRUD and Reorder Monitoring

Purpose: Describes the operation selection, validation, database write, and automatic reorder evaluation sequence.



Flowchart 3 — JSON Backup and Restore

Purpose: Describes both the export path (backup) and import path (restore) including schema validation, duplicate handling, and error recovery.



IV. INDIVIDUAL CONTRIBUTIONS

Arcilla, EJ – Responsible for the system implementation using C# Windows Forms and Entity Framework Core with SQLite. Developed the application code, integrated the database, and compiled the Appendices section, including system screenshots and supporting materials.

Berdejo, Justine Marlowie – Developed the Use Case Diagram, Data Flow Diagrams, and Algorithm Flowcharts to represent system processes and logical operations.

Bitara, Peter John – Designed the system architecture using a Three-Tier (N-Tier) structure. Created the N-Tier Architecture Diagram and defined the responsibilities of each layer. Designed the Entity Relationship Diagram (ERD), including primary and foreign key relationships and database schema organization. Also prepared the Individual Contributions section of this document.

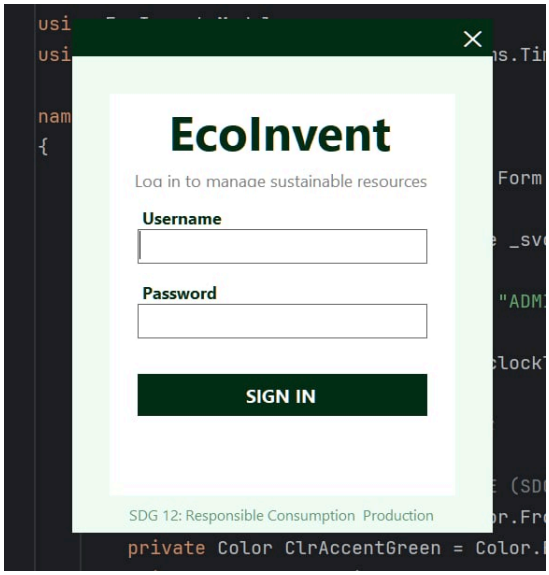
Guico, Karlo Emanuel – Developed the Requirements and Analysis section, including the Functional Requirements, Non-Functional Requirements, and Complexity Analysis. Identified system specifications and operational constraints.

King, Allen Marlon – Prepared the Introduction section, including the Project Overview and Problem Statement. Defined the system objectives and its alignment with United Nations Sustainable Development Goal 12: Responsible Consumption and Production.

V. APPENDICES

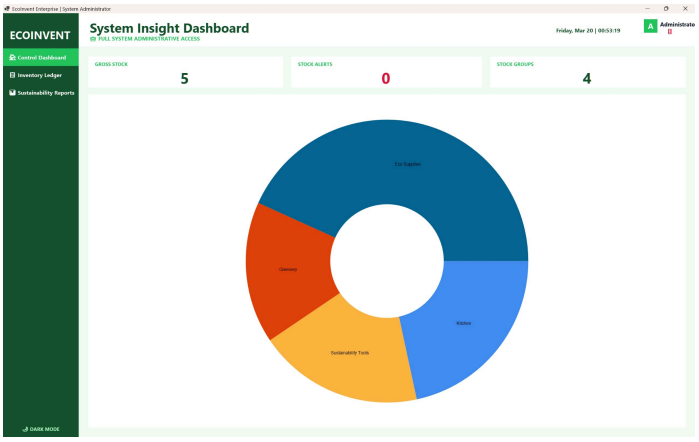
A. Login Interface

The login page allows users to securely access the system using their registered credentials. The system supports role-based authentication, distinguishing between Administrator and Viewer users.



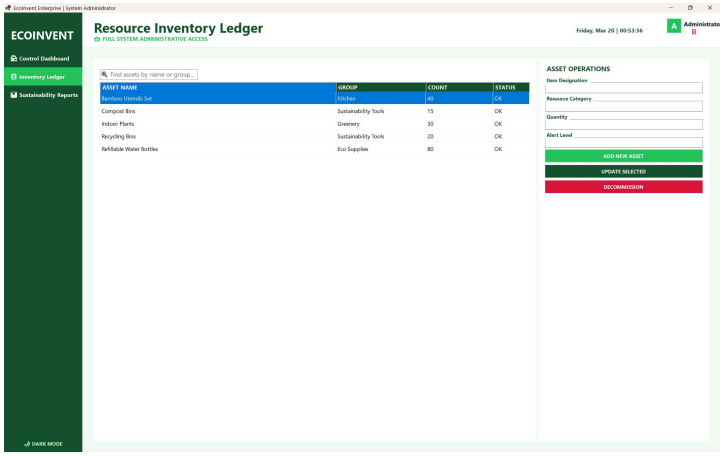
B. Admin Dashboard

The Admin Dashboard provides an overview of the system, including key metrics such as total resources, low stock items, and system insights. It also displays graphical charts to help administrators analyze inventory distribution



C. Inventory Ledger

The Inventory Ledger enables administrators to manage inventory items. Users can perform CRUD operations such as adding new assets, updating existing records, and decommissioning items. The table provides a structured view of all inventory data.

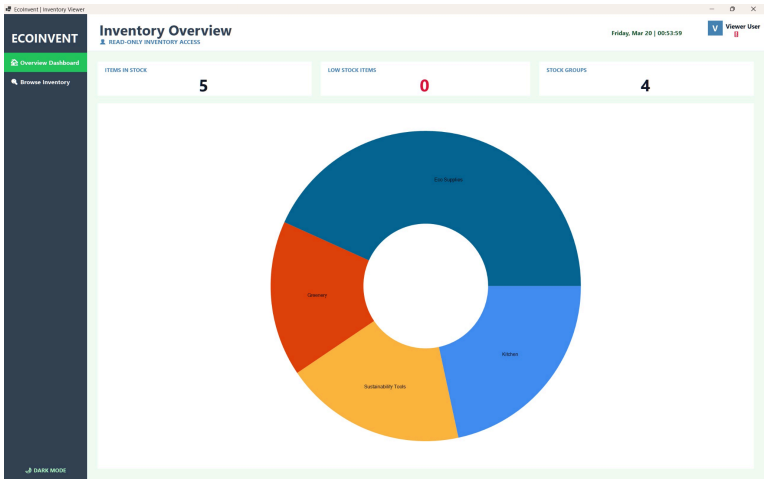


D. Sustainability Reports

The Sustainability Reports module presents data visualization through charts. It helps users analyze resource usage, lifecycle distribution, and sustainability trends to support better decision-making aligned with SDG 12.

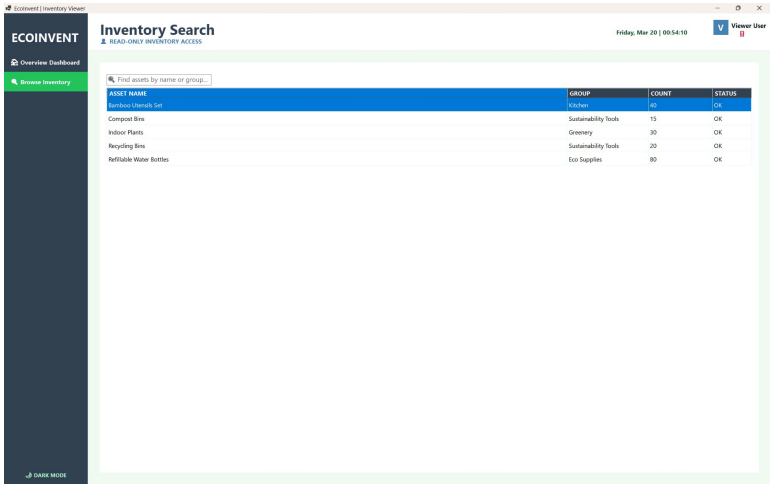
E. Viewer Dashboard

The Viewer Dashboard provides a simplified interface for users with limited access. It allows viewing of inventory data and reports without modification privileges, ensuring data integrity and controlled access.



F. Search and Resource Tracking

The system includes a search functionality that allows users to quickly find inventory items. This improves usability and efficiency when managing large amounts of data.



G. System Features Summary

The EcoInvent system integrates multiple functionalities including secure login, inventory management, reporting, and data visualization. These features collectively support efficient resource management and promote responsible consumption.